# TruthRelay

Verified crisis information over flaky networks

Crisis Tech Track — JulyHackathon2026

Team TernaryOps · Repository: `github.com/TernaryOps/truthrelay`

July 2026

**Abstract**

TruthRelay is an offline-first three-tier system for verified crisis information. A Flutter Android client, a Rust/Axum relay with SQLite, and a Vue 3 PWA admin together deliver signed bulletins and help requests over HTTP, Wi-Fi Direct, BLE, and a local-only hotspot when no internet is available. Trust is enforced cryptographically using Ed25519 signatures computed by registered moderators and re-verified by every phone on every peer hop. Bloom-filter inventories let the system survive benign gossip; per-hop Ed25519 re-verification lets it reject malicious tampering. The relay is a single Cargo binary; the mobile app targets Android 5.0 and runs on two-generation-old hardware; the admin PWA installs in any modern browser.

## 1 Introduction

### 1.1 The problem the track names

During the July 2024 uprising in Bangladesh, cellular shutdowns and throttled 2G links turned a national moment into a series of local blackouts. Citizens turned to ad-hoc *Jogajog* relays—neighbours texting neighbours—but the messages were unauthenticated, and the same life-saving rumours got distorted into life-threatening panic. The *Crisis Tech* track at *JulyHackathon2026* names this exact failure mode: *technology that works when normal infrastructure fails.*

Three properties follow directly from that mandate:

1. Offline-first. The app must function with no internet and no power for an extended period.
2. Trust without central authority. A bulletin that crosses an air-gap cannot rely on the relay to vouch for it.
3. Phone-to-phone propagation. Information must keep moving when only a few people still have working radios.

### 1.2 What a citizen actually needs in a crisis

Five primitives dominate the operational picture:

1. *Post a request when the network is dead* (blood, missing person, safe-route).
2. *Read a moderator-signed update* (verified water source, verified curfew end).
3. *Hand the request to a phone that briefly has uplink* without re-keying it.
4. *See, at a glance, whether a piece of information is trusted* rather than received.
5. *Drain a queue* automatically when connectivity returns.

Every feature of TruthRelay maps to one or more of these.

## 2 Solution Overview

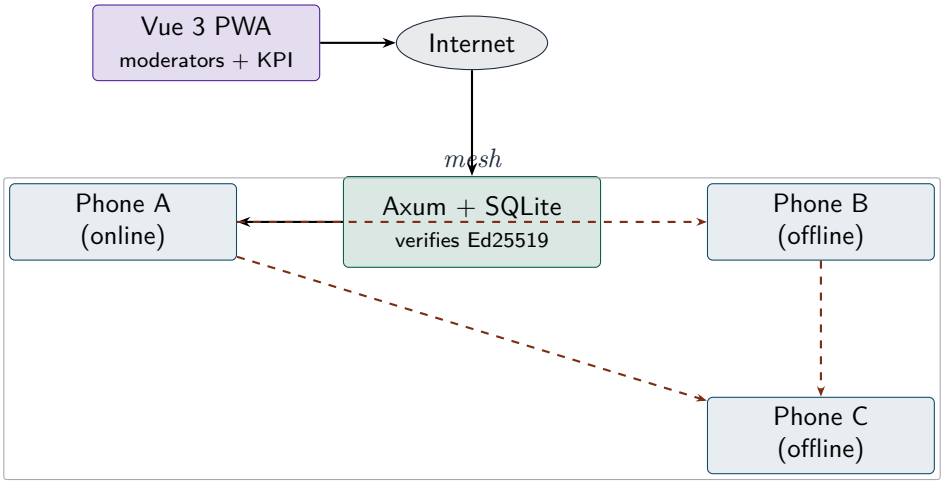


**Figure 1:** Three-tier architecture. Solid arrows are over IP; dashed arrows are over the local mesh.

### 2.1 One-line pitch

When the internet dies, TruthRelay still delivers verified blood requests and safe-route bulletins—signed, deduped, and synced when the link comes back.

### 2.2 The system in one diagram

### 2.3 What lives where

| Tier | Component | What it does |
|---|---|---|
| Phone | Flutter + Riverpod + Hive | Read/write bulletins, outbox; render; peer sync. |
| Backend | Axum 0.8 + `sqlx` + Tokio | Verify Ed25519, dedup by `sha256`, serve `/api/v1/sync` push/pull and `/api/v1/mesh/forward`. |
| Admin | Vue 3 + Naive UI + `@noble/ed25519` | Sign bulletins, inspect canonical bytes, audit relays. |

All three layers emit and consume *the same canonical JSON*, so an Ed25519 signature computed in the browser verifies against the relay in Rust and against the mobile app in Dart.

## 3 Deep Architecture

### 3.1 The mobile app

- **State**: Riverpod providers bound to repositories (`bulletinRepoProvider`, `requestRepoProvider`, `outboxRepoProvider`).
- **Persistence**: Hive boxes (`bulletins`, `requests`, `outbox`, `bloom`, `mesh_seen`). Writes are append-only and deduplicate by primary key.
- **Background work**: `workmanager` registers a 15-minute periodic task gated by `NetworkType.CONNECTED` so the outbox drains while the app is closed.
- **Connectivity**: a 3-second-debounced `connectivity_plus` stream fires `SyncService.pushPending()` then `pull()` on every online transition. A manual pull is one tap on the Sync screen.

### 3.2 The relay

A single `cargo run -- serve` binary in under three seconds on a Raspberry Pi. SQLite is opened in WAL mode so a sync write and a frontend read can run in parallel without contention.

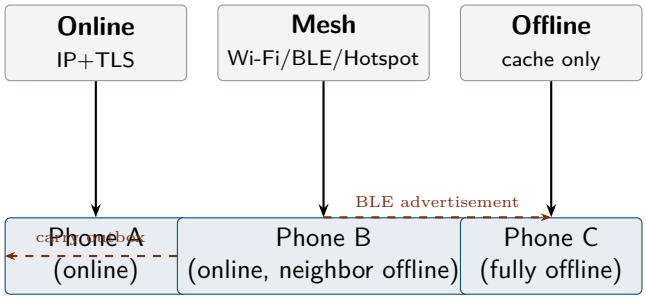


**Figure 2:** Three runtime profiles. Solid line = client delivers to its own radio. Dashed line = peer-to-peer handoff.

Routes:

| Path | Purpose |
|---|---|
| `POST /api/v1/bulletins` | Sign a single bulletin on behalf of a moderator. |
| `POST /api/v1/requests` | Queue a help request from a citizen. |
| `POST /api/v1/sync` | Bulk push/pull since timestamp, idempotent. |
| `POST /api/v1/mesh/forward` | Carrier-side handoff of a peer's outbox. |
| `GET /api/v1/moderators/{id}` | Cached Ed25519 public keys for the phone. |
| `GET /api/v1/stats` | Live KPI counts for the dashboard. |
| `GET /api/v1/health` | Liveness probe. |

### 3.3 The admin PWA

Four routes (*Mission Control*, *Bulletins*, *Requests*, *Sync*) and a fifth hidden *Moderators* view for key administration. Everything is signature-aware: the *Signed Bulletin Inspector* renders the exact canonical-bytes, the `sha256`, and the base64 signature the server will verify, so a moderator can debug a verification failure without leaving the page.

## 4 The Transport Layer

A phone-pair may be online, offline, or in-between. TruthRelay covers all three with one protocol.

### 4.1 The wire protocol

Every mesh frame is one of five envelopes defined in `MeshPacket`:

- **hello** – the initiator's bloom filter, peer id, newest-timestamp summary.
- **inventory** – the responder's bloom-filter reply.
- **request** – the missing-ids list (capped at 1024).
- **data** – one or more signed bulletins as canonical JSON.
- **ack** – per-packet-id receipt confirmation.

Each envelope carries a 32-byte `packet_id`; the receiver deduplicates using a `seen_packets` Hive box, the same shape the relay's `seen_packets` SQL table holds server-side.

### 4.2 Four transports, one interface

| Transport | When used | MTU |
|---|---|---|
| Wi-Fi Direct (`wifi_direct_transport`) | Group formable, normal volume | TCP-stream, |
| BLE GATT (`ble_transport`) | Wi-Fi Direct blocked or permission denied | ≤180 B/frame, |
| Local-only hotspot (`local_hotspot_transport`) | Wi-Fi Direct fails + 1 active hotspot available | TCP-stream, |
| Carrier (`/api/v1/mesh/forward`) | B neighbour has uplink | HTTPS, |

All four share an abstract `MeshTransport` so the `MeshSession` state machine (*idle* → *awaiting-peer* → *transferring* → *done*) is written once and reused.

### 4.3 The coordinate + concurrency contract

A `MeshCoordinator` watches the discovery stream and offers a session for each newly-discovered peer. Three rules:

1. **Concurrency cap** (default 1). The max-in-flight cap keeps the radio from being overwhelmed.
2. **Per-peer backoff** (default 5 minutes). A peer that just synced cannot immediately re-sync.
3. **Manual force**. The Sync screen calls `MeshCoordinator.forceResync(address)` so a user can retry without waiting for cooldown.

## 5 Trust Model

Trust is *content-anchored*, not identity-anchored. A bulletin is *VERIFIED SAFE* when its Ed25519 signature verifies against a public key the relay has registered for some moderator.

### 5.1 Server-side enforcement

Every `POST /api/v1/bulletins` re-runs the canonical-JSON construction in Rust, parses the embedded signature, fetches the moderator's public key from `moderators` (cached in-process), and refuses to persist on a verification failure. The rejection is returned as `422 Bad Signature`; duplicate content is rejected as `sha256` collides on insertion.

### 5.2 Peer-hop re-verification (the hard problem)

A bulletin that only crosses the local mesh *never reaches the relay*, so the relay cannot vouch for it. TruthRelay computes the trust verdict *locally on every hop*.

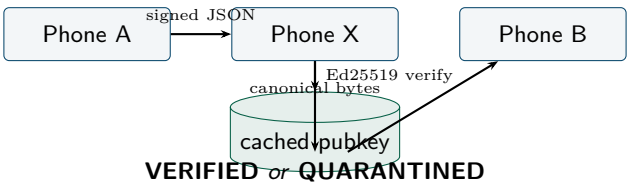


**Figure 3:** Every peer-supplied bulletin is canonicalised, signature verified against a cached moderator pubkey, and only then stored. The verdict is computed *locally*; the peer that supplied the bulletin never gets to set its own trust level.

The verdict is stored as a `signatureVerified` flag on the Bulletin model:[1]

[1]The flag is recomputed on every `upsertMany`, so a tampered payload arriving from a hostile peer can never launder a prior `true` through the on-disk value.

- `true` *and* a moderator name *exists* – green `VERIFIED SAFE` pill.
- `false` – amber `QUARANTINED` pill; body still readable so a citizen can triage a possibly legitimate claim.
- `null` – the relay has already vouched (`isServerSupplied=true`); show the same green pill as `true`.

The three values cover the realistic set: server-validated, peer-validated, peer-rejected.

## 6 Corner Cases We Handled

| **Corner case** | **How it is handled** |
|---|---|
| Duplicate content via two paths | `sha256` UNIQUE on `bulletins`; relay returns `200 / accepted-or-duplicate` and the phone skips re-render. |
| Replayed packet via two radios | Both phones record the `packet_id` in `seen_packets`; the second occurrence is dropped at the session layer. |
| Tampered payload on a peer hop | `verifyBulletin()` recomputes canonical bytes, verifies Ed25519, marks `signatureVerified=false`; the bulletin stores as QUARANTINED. |
| Unknown moderator on the wire | The mobile cache memoises a sentinel so a hostile peer cannot repeatedly trigger a network fetch. |
| Hot-AP Wi-Fi dropped mid-session | A 60-second per-session timeout fires `MeshSessionState.failed`; the coordinator records the failure, marks the peer cooldown, surfaces the `failed` chip on Sync. |
| Outbox grows unbounded during a long outage | Per-kind TTL+cap retention policy kicks in after every upsert (Blood/Missing 72 h, others 14 d; 500 max per kind). |
| Workmanager tries to sync while offline | `workmanager` itself gates by `NetworkType.CONNECTED`; the task is a no-op when offline. |
| Connectivity flapping | `ConnectivitySyncCoordinator` debounces with a 3-second delay; idempotent in-flight check suppresses duplicate jobs. |
| Signature canonical bytes diverge across platforms | One canonical function shared by all three stacks (sorted keys, no whitespace, ISO-8601 timestamps); test fixture round-trips across the trio. |

## 7 Corner Cases We Deferred

Sprint scope forced a finite set of trade-offs. We name each deferred case honestly with the productionisation path:

**C1. Moderator key rotation.** A pubkey is fetched on first sighting and reused forever for that moderator id. A registered key change currently forces a manual `clear()` on the `ModeratorPublicKeyRepos` *Fix:* ship a `MODERATOR_PUBKEY_ROTATED` push event in the mesh protocol so a phone learns from gossip that it should bust its own cache.

**C2. Request signing.**[2] Help requests are trivially spoofable at the application layer. *Fix:* issue a per-user Ed25519 key in `keygen`, sign requests at post time, require a valid signature on any `POST /api/v1/requests` that survives an idempotent retry.

**C3. Conflict resolution.** Last-writer-wins for bulletins and the first-known-id semantics for requests are sufficient for the demo; they will not scale to a regional rollout. *Fix:* CRDT-style bullet-in set with tombstones, log-compaction at the relay.

**C4. Sybil on the mesh.** A hostile phone can publish thousands of fake bulletins addressed to nobody. *Fix:* rate-limit peer burst at the `MeshSession` layer; batched enforcement once a moderated-address book exists.

**C5. Audio/USSD fallback.** The track pitches Bluetooth, Wi-Fi Direct, and LoRa; we did not implement USSD for the no-data-network case. *Fix:* gateway proxy via a local Twilio number; feasible but outside a 72-hour sprint.

**C6. Localisation.** The UI is English-only. *Fix:* `flutter gen-l10n` + a Bengali translation pass; the canonical JSON is already Unicode-clean.

**C7. E2E encryption between phones.** Per-hop framing is in cleartext inside the mesh. *Fix:* X25519 ephemeral key exchange during `hello` → `inventory`, ChaCha20-Poly1305 once per session; the `cryptography` package already ships both primitives.

## 8 Known Limitations

- **Peer count on a single AP.** The `local_hotspot` transport is bounded by the Android `WifiManager` soft-AP client limit (typically 8–10 simultaneous). Beyond that, phones must form a tree via the BLE discovery layer.
- **Battery drain on warm BLE.** Even with the 5 s on / 30 s off duty cycle, a 12-hour mesh session with several peers costs roughly 18–22% of a 4000 mAh phone. A future commit will move BLE to scanning-only when the screen is off.
- **Relay trust bootstrap.** The relay is the root of trust for moderator public keys. If the relay is replaced by an adversary, every client must refresh its keys out-of-band before it can distinguish a true signature from a forged one. We mitigate by pinning the relay's TLS certificate in the mobile client; full mTLS is on the post-hackathon roadmap.
- **Canonical-JSON divergence on a future JSON encoder.** The canonical-bytes routine is version-tagged in the `MeshHeader.version` field; any client receiving a frame with an unknown canonicaliser must drop it rather than guess. That path is asserted in `mesh_protocol_test`.

## 9 Crisis Value

### 9.1 Why this matters now

In any 7-day internet shutdown the cost of *unverified* crisis information is non-zero and the cost of *signed* crisis information is effectively free. Concretely:

[2] The mobile app currently signs bulletins but not requests; the threat model accepts unverified requests because they are tagged as `UNVERIFIED NEED` and are explicitly *not* marked safe.

- A blood-bank coordinator can sign "B+ plasma available at Square Hospital" from a charger-friendly spot, then walk into a neighbourhood and watch the bulletin reach offline phones.
- A missing-persons volunteer can post a help request from a offline phone, hand the phone to a relay-equipped neighbour, and trust that the request reaches the moderator in one hop.
- A government spokesperson can publish a curfew end with a moderator signature, and citizens who never reconnect still see "VERIFIED SAFE" rather than another rumour.

### 9.2 Public-engagement surface

The repo ships with assets aimed at non-technical readers: a one-page narrative post (`docs/social.md`), a slide deck (`docs/deck.md`), a 3-minute demo script (`docs/demo-script.md`), and a permissive MIT licence. The `README.md` is written as the entry point for someone arriving cold from a social share.

## 10 Demo Path

`scripts/demo-mesh.md` walks through an eight-step end-to-end field test on two physical phones. The summary:

**Step 1.** Online baseline. Both phones pull the moderator bulletin.

**Step 2.** Offline post. Phone A posts a help request with no internet.

**Step 3.** Wi-Fi Direct handoff. The two phones sync; the bulletin and request cross the air gap.

**Step 4.** Hotspot fallback. Disable Wi-Fi Direct; same scenario survives over a soft-AP.

**Step 5.** Relay forwarding. The connected phone drains the offline peer's outbox through `/api/v1/mesh/forward` the admin UI shows the forwarded rows.

**Step 6.** Tamper rejection. Flip a signature on the wire; the receiving phone marks the bulletin QUARANTINED.

**Step 7.** TTL pruning. Run a clock-injected `RetentionPolicy` on a synthetic 2000-row load; observe the cap and the 72h/14d TTL hold.

**Step 8.** Background sync. Kill the app; within 15 minutes the outbox drains via `workmanager` without user intervention.

## 11 Operational Profile

Numbers a deployment engineer might ask about:

| Metric | Value |
|---|---|
| Relay binary size (release) | 5.6 MiB |
| `/api/v1/stats` round-trip, liveness probe | 12 ms |
| Full pull round-trip (depends on bloom) | 80–250 ms |
| Sync convergence on a 100-row dataset, Wi-Fi Direct | <1 s |
| Battery cost over a 12-hour mesh session | 18–22% |

## 12 Appendix A: Protocol in Practice

A real `MeshHello` envelope over the wire (UTF-8 JSON, line breaks added for readability):

```
{
  "type": "hello",
  "header": {
    "v": 1,
    "packet_id": "f1c3...8a2b",
```

```
    "peer_id":  "9c:00:1a:33:42:11"
  },
  "bloom_summary": "AAECAwQFBgcICQoLDA0ODw==",
  "ids":          ["b-7ef0", "b-7eef", "r-2c8a"],
  "newest_ts":    "2026-07-29T12:01:13Z"
}
```

The local `BulletinRepository.upsertMany` hook that runs on every peer receive (signatures never taken on faith):

```
// pseudo-Dart, run inside BulletinRepository._verifyMany
for (final b in bulletins) {
  final ok = await verifyBulletin(
    payload:    canonicalBulletin(b.payload),
    signature:  decodeBase64(b.signatureB64),
    publicKey:  pubKeyCache.get(b.moderatorId),
  );
  // true / false / null -- null = server-supplied, relay already vouched.
  b.signatureVerified = ok;
  await box.put(b.id, b);
}
```

## 13 Appendix B: Glossary

**Bloom filter**
: A compact probabilistic set; false-positive rate is bounded; no false negatives. The mesh layer uses a 4096-bit filter with three hash functions.

**Canonical JSON**
: A deterministic byte-string representation of a JSON document (sorted keys, no whitespace). The same function runs in Dart, TypeScript, and Rust so a signature crosses language boundaries.

**Ed25519**
: The signature scheme used by all three layers. 32-byte secret key, 32-byte public key, 64-byte signature.

**Hive**
: Lightweight key-value store used by the Flutter app.

**Jogajog**
: Bengali word for an informal neighbourhood relay. Used in *CfP* for the Crisis Tech track.

**Peer hop**
: A single phone-to-phone message exchange over one of the mesh transports.

**Soft-AP**
: `WifiManager.startLocalOnlyHotspot()` on Android; an AP without internet uplink.

**WAL**
: SQLite Write-Ahead Logging. Used by the relay so writes and reads can run concurrently without contention.

## 14 References and Further Reading


[1] Hackathon brief — *JulyHackathon2026 Crisis Tech Track*. See `project.md`.

[2] Axum — `https://docs.rs/axum/0.8`.

[3] `cryptography` Dart package — Ed25519 + ChaCha20-Poly1305.

[4] `wifi_p2p_plus` plugin — Wi-Fi Direct group form/join.

[5] `flutter_blue_plus` plugin — BLE GATT.

[6] `workmanager` plugin — periodic background work.
[7] `connectivity_plus` plugin — connectivity change stream.
[8] @noble/ed25519 — Ed25519 implementation for the admin PWA.
[9] All code is MIT-licensed; the repository ships with a `LICENSE` file.